

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO_2 ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code: CSA0609

Course Title: Design and Analysis of Algorithms

Name : Jeeva

Reg No: 192421334

Q14. Debugging Binary Search (Infinite Loop Issue)

The Binary Search code below leads to an infinite loop for some inputs. Identify the logical errors in updating search boundaries. Explain why the loop fails to terminate. Debug the algorithm to correctly adjust low and high indices. Optimize mid calculation if necessary. Analyze complexity and rewrite the corrected version.

```
def binary_search(arr, key):  
    low, high = 0, len(arr)-1  
    while low <= high:
```

```
        mid = (low + high)//2
```

```
        if arr[mid] == key:
```

```
            return mid
```

```
    elif arr[mid] < key:
```

```
        low = mid # ERROR
```

```
    else:
```

```
        high = mid # ERROR
```

FIRST ERROR:

```
1- def binary_search(arr, key):
2-     low, high = 0, len(arr)-1
3-     while low < .high:
4-         mid = (low + high)//2
5-         if arr[mid] == key:
6-             return mid
7-         elif arr[mid] < key:
8-             low = mid
9-         else:
10-             high = mid
11
```

Enter Input here

If your code takes input, add it in the above box before running

Output

Status : Runtime error

Time: 0.0200 secs Memory: 10.88 Mb

Runtime Error

SIGHUP

Error

File "/mnt/sol.py", line 3
while low < .high:
 ^
SyntaxError: invalid syntax

Incorrect Code

```
while low < high:
```

Correct Code

```
while low <= high:
```

Identify the Error

The loop condition is incorrect because it uses `<` instead of `<=`. Binary Search should continue as long as the lower index is less than or equal to the higher index.

Effect of the Error

The algorithm stops before checking the last remaining element in the search range. As a result, the program may incorrectly report that the key is not found even when it is present in the array. This reduces the accuracy of the search algorithm.

Explanation

In Binary Search, the search space becomes smaller after every iteration. Eventually, only one element remains, where `low` and `high` become equal. This final element must also be checked. Using `while low <= high` ensures that every possible element is examined before terminating the search.

SECOND ERROR:

```
1 def binary_search(arr, key):
2     low, high = 0, len(arr)-1
3     while low <= high:
4         mid = (low + high)//2
5         if arr[mid] == key:
6             return mid
7         elif arr[mid] < key:
8             low = mid.
9         else:
10            high = mid
11
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Runtime error

Time: 0.0200 secs Memory: 10.988 Mb

Runtime Error

SIGHUP

Error

File "/mnt/sol.py", line 8
low = mid.
 ^
SyntaxError: invalid syntax

Incorrect Code

low = mid

Correct Code

low = mid + 1

Identify the Error

The lower bound is updated incorrectly after checking the middle element. The statement `low = mid` does not remove the already checked middle element from the search range.

Effect of the Error

The search interval may not reduce properly, causing the algorithm to repeatedly examine the same middle element. This can result in an infinite loop or prevent the program from finding the correct element. The search process becomes inefficient and may never terminate.

Explanation

When `arr[mid] < key`, it means the key cannot be at the middle position because that element has already been compared. Therefore, the search should continue from the next position by updating `low = mid + 1`. This correctly eliminates the left half, including the middle element, and ensures that the search range becomes smaller in every iteration.

THIRD ERROR:

```
1 def binary_search(arr, key):
2     low, high = 0, len(arr)-1
3     while low <= high:
4         mid = (low + high)//2
5         if arr[mid] == key:
6             return mid
7         elif arr[mid] < key:
8             low = mid+1
9         else:
10            high = mid.
11
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Runtime error

Time: 0.0400 secs Memory: 10.748 Mb

Runtime Error

SIGHUP

Error

File "/mnt/sol.py", line 10
high = mid.
^
SyntaxError: invalid syntax

Incorrect Code

high = mid

Correct Code

high = mid - 1

Identify the Error

The upper bound is updated incorrectly after comparing the middle element with the key. The statement `high = mid` keeps the middle element within the search range even though it has already been checked.

Effect of the Error

The search range may fail to shrink, causing the algorithm to repeatedly check the same element. This can lead to an infinite loop or incorrect search results. The program may fail to terminate or return the wrong answer.

Explanation

When `arr[mid] > key`, the required element must be in the left half of the array. Since the middle element has already been checked, it should not be included in the next search. Therefore, the correct update is `high = mid - 1`. This removes the middle element from the search range, allowing the algorithm to continue correctly and efficiently.

CORRECTED CODE:

```
def binary_search(arr, key):
```

```

    low = 0    high =
len(arr) - 1    while
low <= high:

    mid = (low + high) // 2

    if arr[mid] == key:
        return mid

    elif arr[mid] < key:
        low = mid + 1    else:
        high = mid - 1    return
-1

```

The screenshot shows a Python IDE with a dark theme. The editor displays a function `def binary_search(arr, key):` with the following logic:

- Initialize `low = 0` and `high = len(arr) - 1`.
- Enter a `while low <= high:` loop.
- Calculate `mid = (low + high) // 2`.
- If `arr[mid] == key`, return `mid`.
- Else if `arr[mid] < key`, update `low = mid + 1`.
- Else, update `high = mid - 1`.

 The right sidebar shows the 'Run' button and 'Visualize Code' option. Below, the 'Output' section displays:

- Input field: 'Enter Input here'.
- Instruction: 'If your code takes input, add it in the above box before running.'
- Status: 'Status : Successfully executed'.
- Performance metrics: 'Time: 0.0400 secs' and 'Memory: 8.512 Mb'.
- Footer: 'No details to show'.

BINARY SEARCH ALGORITHM

ALGORITHM

1. Start.
2. Read the sorted array and the key element to be searched.
3. Initialize `low = 0` and `high = n - 1`.
4. Repeat while `low <= high`:
 - Calculate the middle index:

`mid = (low + high) // 2`

- If $\text{arr}[\text{mid}] == \text{key}$, return the index mid.
- If $\text{arr}[\text{mid}] < \text{key}$, update $\text{low} = \text{mid} + 1$ and search in the right half.
- Otherwise, update $\text{high} = \text{mid} - 1$ and search in the left half.

5. If the loop ends without finding the key, return -1.

6. Stop.

Step-by-Step Manual Analysis

Initial State

Array : [10, 20, 30, 40, 50]

Key : 50

$\text{low} = 0$ $\text{high} = 4$

Iteration 1

- Calculate middle index:

$$\text{mid} = (0 + 4) // 2 = 2$$

- $\text{arr}[2] = 30$
- Since $30 < 50$, search the right half.
- Update: $\text{low} = \text{mid} + 1 = 3$

Current search range:

[40, 50]

Iteration

- Now: • $\text{low} =$

$\text{high} =$

- Calculate:

$$\text{mid} = (3 + 4) // 2 = 3$$

- $\text{arr}[3] = 40$
- Since $40 < 50$, search the right half.

6

7

7

- Update: $\text{low} = \text{mid} + 1 = 4$

Current search

range:

[50]

Iteration 3

- Now:
- $\text{low} = 4$

$\text{high} = 4$

- Calculate: $\text{mid} = (4 + 4) // 2 = 4$
- $\text{arr}[4] = 50$
- Since $\text{arr}[\text{mid}] == \text{key}$, the search is successful.

Return:

Index = 4

Output

Element found at index: 4

EXPLANATION

Binary Search is an efficient searching algorithm that works only on sorted arrays. Instead of checking every element one by one, it compares the key with the middle element of the current search range. Based on the comparison, it discards one half of the array and continues searching only in the remaining half. This process repeats until the key is found or the search range becomes empty. Because the search space is reduced by half in every iteration, Binary Search requires far fewer comparisons than Linear Search, making it highly efficient for large datasets.

OUTPUT

Element found at index: 4

Time Complexity: $O(1)$
